Os Pilares da Excelência em ABAP Moderno

Dominando Robustez, Performance e Design no S/4HANA



Nesta jornada, vamos além da sintaxe. Exploraremos os três princípios fundamentais que transformam um desenvolvedor funcional em um verdadeiro arquiteto de software, construindo soluções que não apenas funcionam, mas que são seguras, rápidas e prontas para o futuro.

NotebookLM

A Tríade da Qualidade de Código

No S/4HANA, 'funcionar' não é suficiente. A qualidade de um software profissional se apoia em três eixos de verificação, garantindo conformidade, correção e eficiência.



Conformidade

Ferramenta: ATC (Análise Estática)

Pergunta que Responde: 'O código segue as regras da empresa e da SAP? É seguro? Está pronto para a nuvem?'

Correção

Ferramenta: ABAP Unit (Análise Dinâmica)

Pergunta que Responde: 'O código faz o que a regra de negócio pede? Uma mudança hoje quebrou algo antigo (regressão)?'

Performance

Ferramenta: ABAP Profiling (Análise Dinâmica)

Pergunta que Responde: 'O código é eficiente? Consome recursos excessivos ou faz chamadas desnecessárias ao banco?'

Esses pilares são a base de esteiras de CI/CD modernas, automatizando a qualidade antes do transporte.

NotebookLM

Análise Estática com ATC: O Guardião do Padrão e do Clean Core

O ABAP Test Cockpit (ATC) lê seu código sem executá-lo, como um revisor experiente procurando por padrões de má programação, vulnerabilidades e código obsoleto antes que cheguem em produção.

Principais Categorias de Verificação

	Categoria	Descriminação
⚙	**Sintaxe e Modernização**	`MOVE`, `COMPUTE` em vez de expressões inline. `SELECT *` ineficiente.
🚀	**Performance (HANA)**	O **pecado capital:** `SELECT` dentro de `LOOP`. Leitura de tabelas sem `WHERE`.
🛡	**Segurança**	Falta de `*AUTHORITY-CHECK*` antes de operações críticas. Risco de SQL Injection.
🧱	**Robustez**	Código morto (variáveis não usadas). Tipos incompatíveis que podem causar Dumps.
☁	**Cloud Readiness**	Uso de objetos não liberados. Acesso direto ao sistema operacional.

Gerenciando Isenções

```
" Para silenciar um aviso de falso-positivo, usamos pragmas.
" Ex: Ignorar aviso de texto 'hardcoded' em uma variável de teste.
DATA(lv_dummy) = 'Teste' ##NO_TEXT. ←—— O pragma ##NO_TEXT instrui o ATC a ignorar este aviso específico nesta linha.

" Quick Fixes (Ctrl+1 no ADT) podem corrigir muitos erros automaticamente.
```

NotebookLM

Análise Dinâmica com ABAP Unit: Sua Rede de Segurança Contra Regressões

Enquanto o ATC olha a *forma*, o ABAP Unit testa a *função*. É uma coleção de testes automatizados que garante que novas funcionalidades não quebrem as existentes.

Anatomia de um Teste

Define uma classe local visível apenas para o framework de testes.

```
CLASS lcl_test DEFINITION FOR TESTING
  RISK LEVEL HARMLESS
  DURATION SHORT.
...
```

Garante execução rápida para não atrasar a esteira de CI/CD.

Essencial. Garante que o teste não altera dados (não faz `COMMIT`), sendo seguro para automação.

Ciclo de Vida (Fixtures)

Os métodos `setup` e `teardown` garantem o isolamento, criando um ambiente limpo e previsível para cada teste individual.

Código Anotado - Padrão AAA

Preparação do ambiente de teste.

Execução da funcionalidade.

Verificação do resultado.

Testando se o código FALHA corretamente.

" SUCESSO! O erro era esperado. O teste passa."

```
METHOD test_divisao_sucesso.
  " ARRANGE: mo_cut foi criado no setup.

  " ACT: Executa o método a ser testado.
  TRY.
    DATA(lv_result) = mo_cut->dividir( iv_a = 10, iv_b = 2 ).
  CATCH cx_sy_zerodivide.
    cl_abap_unit_assert=>fail( 'Não deveria lançar exceção' ).
  ENDTRY.

  " ASSERT: Verifica se o resultado é o esperado.
  cl_abap_unit_assert=>assert_equals(
    act  = lv_result
    exp  = 5
    msg  = 'Divisão de 10 / 2 deve ser 5'
  ).
ENDMETHOD.

METHOD test_divisao_por_zero.
  " Testando se o código FALHA corretamente.
  TRY.
    mo_cut->dividir( iv_a = 10, iv_b = 0 ).
    cl_abap_unit_assert=>fail( 'Deveria ter lançado exceção' ).
  CATCH cx_sy_zerodivide.
    " SUCESSO! O erro era esperado. O teste passa.
  ENDTRY.
ENDMETHOD.
```

NotebookLM

Escrevendo Código Seguro: Conversões Explícitas e Exceções Ricas

O Perigo da Conversão Implícita

O ABAP clássico tenta "adivinhar" as conversões, mas isso leva a erros silenciosos como truncamento de texto e estouro numérico (Dumps). A robustez exige intenção clara.

Antes e Depois

Antes (Implícito - Risco)

```
DATA lv_num TYPE i.
lv_num = '100.9'.

// Arredonda para 101 sem aviso!
```

Depois (Explícito - Seguro)

```
DATA(lv_num) = CONV i( '100.9' ).

// Deixa claro a intenção. Envolva
em TRY...CATCH.
```

Tratamento de Erros Moderno

Esqueça sy-subrc. Erros de negócio devem ser tratados com objetos de exceção, que carregam contexto, podem ser vinculados a mensagens traduzíveis (T100) e seguem um contrato claro.

Categoria de Exceção	Quando Usar	Contrato
CX_STATIC_CHECK	Erros de negócio previsíveis (Saldo insuficiente, Cliente bloqueado).	O consumidor é **obrigado** a tratar com TRY...CATCH.
CX_NO_CHECK	Erros técnicos graves (Divisão por zero, Ponteiro nulo).	O tratamento é opcional; se não tratado, gera um Dump.

```
RAISE EXCEPTION TYPE zcx_travel_error EXPORTING textid = zcx_travel_error=>negative_amount.
```

NotebookLM

PILAR 2: PERFORMANCE - Otimizando para Velocidade Máxima

A Revolução do Code Pushdown: Leve a Lógica até os Dados

O maior inimigo da performance em HANA é transferir milhões de linhas para o servidor de aplicação apenas para calcular, filtrar ou agregar. A regra moderna é: empurre a lógica para o banco de dados.

ABAP Clássico - Ineficiente

```
SELECT * FROM zrap_travel
INTO TABLE @lt_data.
```

```
LOOP AT lt_data...
  IF..,
    lv_total = lv_total + ...
```



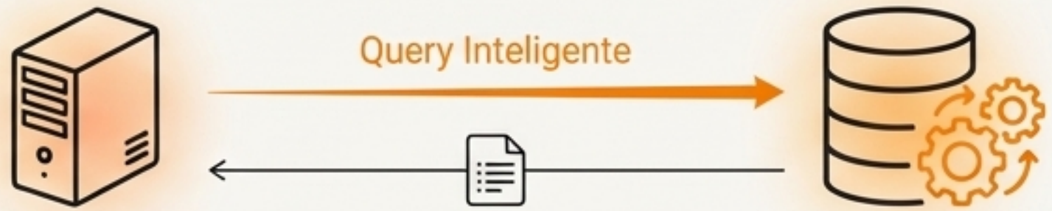
Muito tráfego de rede / Milhões de linhas

Resultado: Poucas linhas agregadas.

ABAP SQL Moderno - Eficiente

```
SELECT ...
SUM(...) ...
CASE ...
```

Processa `WHERE`, `GROUP BY`, `SUM`, etc.



Resultado: Apenas os dados necessários.

A escolha não é entre ABAP SQL e CDS Views, mas entre lógica na aplicação vs. lógica no banco.
Use ABAP SQL para queries específicas e CDS Views para modelos de dados reutilizáveis.

NotebookLM

ABAP SQL Avançado: Expressões, Lógica e Agregações em Uma Só Chamada

O ABAP SQL moderno herdou os superpoderes do CDS, permitindo a construção de relatórios analíticos complexos diretamente na **query**, eliminando o pós-processamento em ABAP.

```
SELECT FROM zrap_travel
  FIELDS
    agency_id,
    currency_code,

    COUNT( * ) AS number_of_travels,
    SUM( total_price ) AS total_amount,

    AVG( total_price AS DEC( 15,2 ) ) AS average_ticket,

    CASE
      WHEN SUM( total_price ) > 100000 THEN 'Platinum Partner'
      WHEN SUM( total_price ) > 10000  THEN 'Gold Partner'
      ELSE 'Standard Partner'
    END AS partner_category

  GROUP BY agency_id, currency_code

  HAVING SUM( total_price ) > 1000

  ORDER BY total_amount DESCENDING
  INTO TABLE @DATA(lt_report).
```

1. AGREGAÇÃO: Contar e somar direto no banco.

2. EXPRESSÃO ARITMÉTICA: Calcular ticket médio com casting.

3. LÓGICA CONDICIONAL: Categorizar parceiros com CASE.

4. FILTRO PÓS-AGREGAÇÃO: Filtrar usando o resultado da soma.

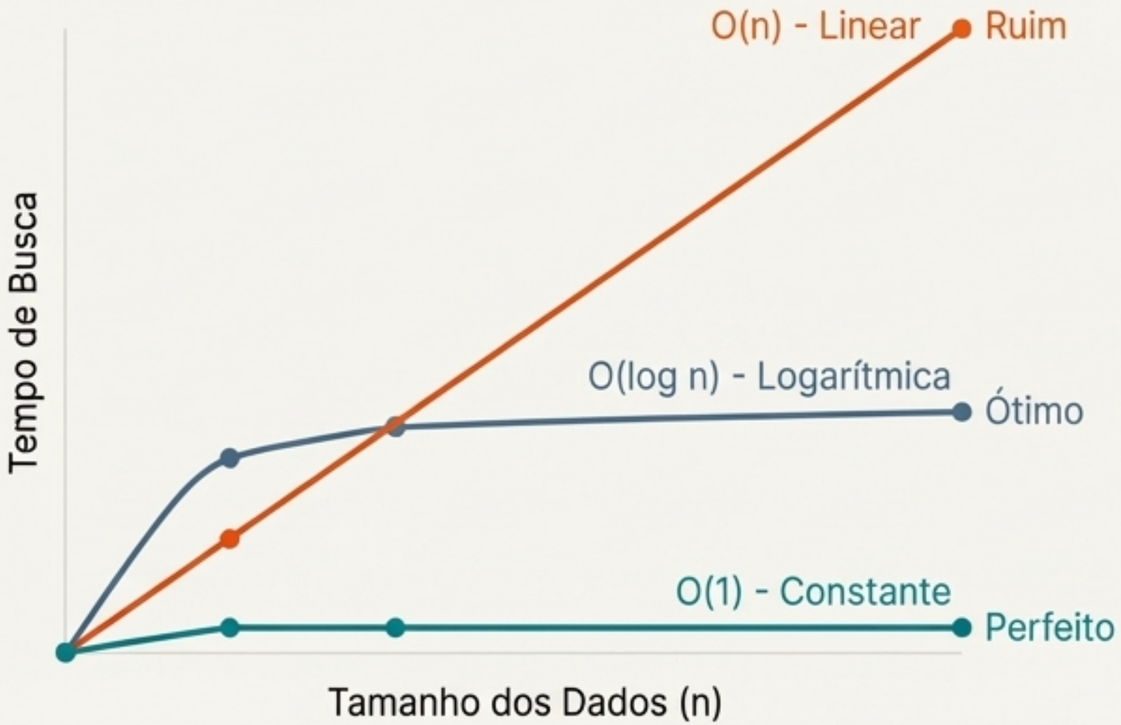
Callout de Robustez: Lembre-se da função `COALESCE( campo, 0 )` em cálculos para tratar valores nulos e evitar que anulem todo o resultado.

NotebookLM

Dominando Tabelas Internas: Do Custo O(n) à Velocidade O(1)

O Custo Oculto da Busca Linear

Fazer `LOOP ... WHERE` em uma Tabela Standard é o erro de performance mais comum. O tempo de busca cresce linearmente com os dados (O(n)), causando timeouts em produção.



A Solução: Chaves Secundárias e Referências

Crie 'índices' em memória com `SECONDARY KEYS` para transformar buscas lineares em logarítmicas ou constantes. Use referências para evitar cópias de memória custosas.

```
TYPES ty_t_data TYPE STANDARD TABLE OF ty_data
                WITH UNIQUE HASHED KEY key_id COMPONENTS id.

" LENTO: Varre a tabela inteira.
LOOP AT lt_data INTO ls_data WHERE id = 500. ... ENDLOOP.

" RÁPIDO: Salta direto para a linha usando o índice hash.
LOOP AT lt_data ASSIGNING <fs_data> USING KEY key_id WHERE id =
  500.
ENDLOOP.
```

ANOTAÇÃO: <fs_data> é um ponteiro, não uma cópia. Alterá-lo modifica a tabela diretamente. Rápido e eficiente.

Destaque: Refatorar um `Nested Loop` (loop aninhado) com chaves secundárias pode resultar em ganhos de performance de mais de 5.000 vezes.

NotebookLM

PILAR 3: DESIGN & ELEGÂNCIA - Refinando a Obra com Inteligência

O Contrato é a Interface, não a Classe

Um sinal de senioridade é programar para interfaces (o "o quê"), não para implementações concretas (o "como"). Isso cria um código desacoplado, testável e flexível a mudanças.

Tabela de Decisão

Característica	Herança (`INHERITING FROM`)	Interface (`INTERFACES`)
Relacionamento	**"É um"** (Cachorro é um Animal) - Rígido	**"Faz algo"** (Cachorro é Latidor) - Flexível
Acoplamento	**Alto/Forte**. Mudanças na mãe quebram os filhos.	**Baixo/Fraco**. A implementação pode mudar, o contrato não.
Multiplicidade	**Não**. ABAP só permite herança simples.	**Sim**. Uma classe pode implementar N interfaces.
Testabilidade	Difícil. Acoplado à classe concreta.	**Fácil**. Permite injetar 'Mocks' que seguem o mesmo contrato.

ABAP

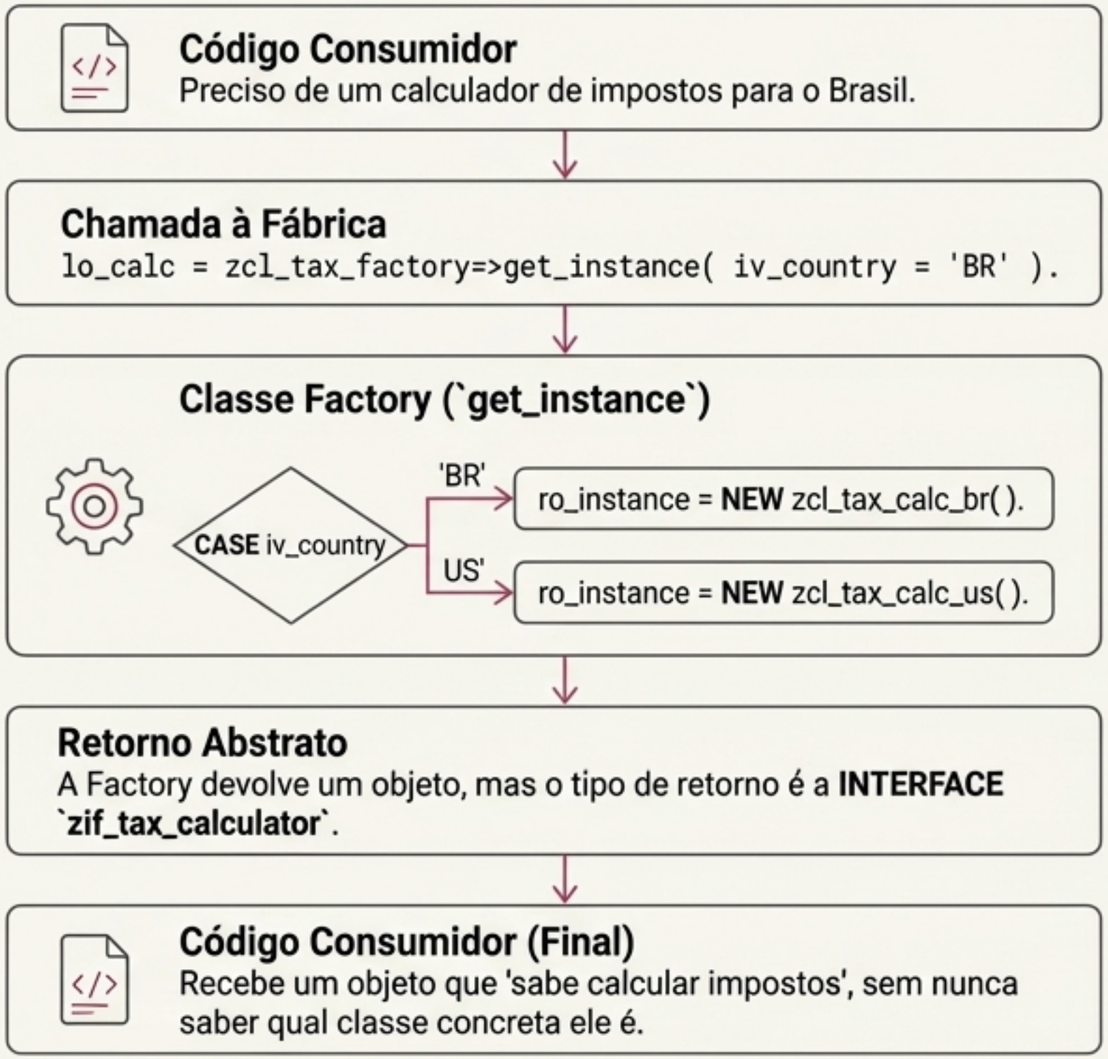
```abap
" RUIM: Acoplado à implementação concreta.
DATA: lo_calc TYPE REF TO zcl_travel_calc_br.
lo_calc = NEW #( ).

" BOM: Desacoplado, depende apenas do contrato.
DATA: lo_calc TYPE REF TO zif_travel_calc.
lo_calc = zcl_travel_factory=>get_calculator( 'BR' ).
```

NotebookLM

Centralizando a Criação de Objetos com o Padrão Factory

Evite espalhar `NEW zcl_minha_classe()` pelo código. Isso cria um acoplamento forte. Uma classe **"Fábrica" centraliza a decisão de qual objeto instanciar, escondendo a complexidade dos consumidores.**



A Fábrica

```
CLASS zcl_travel_factory DEFINITION PUBLIC.
  PUBLIC SECTION.
    CLASS-METHODS get_calculator
      IMPORTING iv_country TYPE land1
      RETURNING VALUE(ro_calc) TYPE REF TO zif_travel_calc.
                                  └──> //<- Retorna a Interface!
ENDCLASS.

CLASS zcl_travel_factory IMPLEMENTATION.
  METHOD get_calculator.
    CASE iv_country.
      WHEN 'BR'.
        ro_calc = NEW zcl_travel_calc_br( ).
      WHEN OTHERS.
        ro_calc = NEW zcl_travel_calc_global( ).
    ENDCASE.
  ENDMETHOD.
ENDCLASS.
```

NotebookLM

Código Expressivo e Autodocumentado

Funções de String Modernas

Substitua comandos antigos e verbosos por funções embutidas e encadeáveis. O código se torna mais legível e conciso.

Operação	Comando Legado (Evitar)	Função Moderna (Usar)
Maiúsculas	`TRANSLATE x TO UPPER CASE`	`x = to_upper( x )`
Busca	`SEARCH x FOR 'ABC'`	`IF contains( val = x sub = 'ABC' )`
Substituir	`REPLACE 'A' WITH 'B' INTO x`	`x = replace( val = x sub = 'A' with = 'B' )`
Concatenar	`CONCATENATE a b INTO c`	`c = |{ a } { b }|`

A Filosofia do ABAP Doc

Bons comentários explicam o *porquê*, não o *o quê*. Use ABAP Doc (`"\"`) para documentar a interface pública (o contrato), ensinando outros desenvolvedores a usar sua classe.

Exemplo de ABAP Doc Profissional

```
\"! <p class="shorttext">Calcula o valor do desconto de fidelidade.</p>
\"! A lógica é baseada no <em>histórico de compras</em> do cliente.
\"!
\"! @parameter iv_customer_id     | ID do Cliente (Tabela KNA1)
\"! @returning rv_discount        | Valor monetário do desconto calculado
\"! @raising   zcx_customer_error | Disparado se o cliente estiver bloqueado
METHODS calculate_discount
   IMPORTING iv_customer_id     TYPE kunnr
   RETURNING VALUE(rv_discount) TYPE kbetr
   RAISING   zcx_customer_error.
```

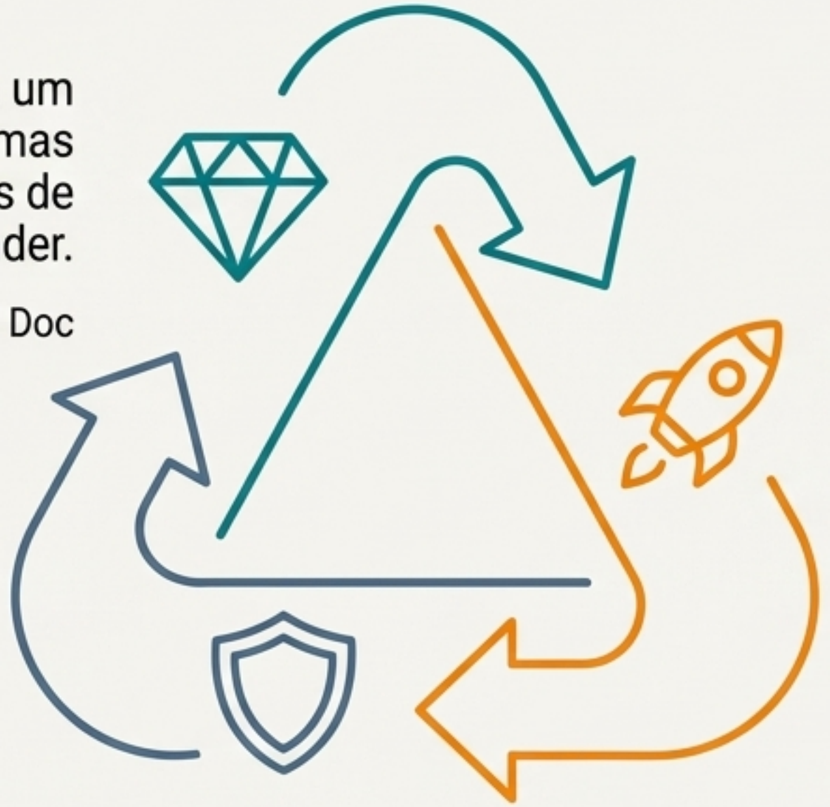
Pressione F2 no ADT para ver esta documentação instantaneamente, sem precisar navegar para a classe.

NotebookLM

O Artesão de Software Moderno

Por fim, refinamos a obra com um design inteligente, criando sistemas flexíveis, desacoplados e fáceis de manter e entender.

Interfaces, Padrões de Projeto, ABAP Doc



Em seguida, otimizamos a máquina, empurrando a lógica para o banco de dados e gerenciando a memória com precisão cirúrgica.

ABAP SQL, Code Pushdown, Chaves Secundárias

Começamos com um alicerce inabalável, garantindo que nosso código é correto, seguro e testável desde o primeiro dia.

ATC, ABAP Unit, Exceções, Segurança.

Adotar estes três pilares não é sobre seguir regras, é sobre mudar a mentalidade. É a transição de um programador que entrega funcionalidades para um artesão que constrói ativos de software duradouros e de alta qualidade.

NotebookLM